

需求规格说明书

Software Requirements Specification

基于轻量化卷积神经网络的 工业零部件表面缺陷智能检测系统

文档版本	V1.0
项目名称	工业零部件表面缺陷智能检测系统
课程名称	制造智能技术
开发方式	Vibe Coding (AI 辅助人机协同开发)
编制日期	2026 年 9 月

本文档依据 IEEE 830 标准及 GB/T 8567-2006《计算机软件文档编制规范》编写，对系统的功能需求、非功能需求、接口需求和数据需求进行完整、准确、无歧义的描述。

目录

1 引言

- 1.1 编写目的
- 1.2 项目背景
- 1.3 术语与缩略语
- 1.4 参考资料

2 任务概述

- 2.1 系统目标
- 2.2 用户特征
- 2.3 运行环境
- 2.4 条件与约束

3 功能需求

- 3.1 功能划分
- 3.2 缺陷检测功能
- 3.3 历史记录管理功能
- 3.4 统计分析功能
- 3.5 系统管理功能
- 3.6 用例描述

4 非功能需求

- 4.1 性能需求
- 4.2 安全需求
- 4.3 可用性需求
- 4.4 可维护性需求
- 4.5 可移植性需求

5 接口需求

5.1 用户接口

5.2 软件接口

5.3 硬件接口

6 数据需求

6.1 数据字典

6.2 数据库设计

6.3 数据采集与处理

7 其他需求

7.1 配置管理需求

7.2 文档需求

7.3 培训需求

1 引言

1.1 编写目的

本文档是"基于轻量化卷积神经网络的工业零部件表面缺陷智能检测系统"的需求规格说明书，旨在对系统的功能需求、非功能需求、接口需求和数据需求进行完整、准确、无歧义的描述。本文档的预期读者包括：项目开发人员（用于指导系统设计与编码实现）、测试人员（用于制定测试计划）、课程指导教师（用于评估需求完整性与合理性）、以及项目维护人员（用于后续系统升级与维护）。

【编写说明】需求规格说明书是课程设计提交物清单中的第二项，老师会重点看需求是否完整、描述是否无歧义、功能是否可验证。本文档严格按照 IEEE 830 标准结构编写，每个需求都有唯一编号，便于追溯和验证。

1.2 项目背景

在现代制造业中，产品表面质量直接影响产品的性能、可靠性和使用寿命。机械制造、汽车零部件、五金冲压等离散制造场景中，工件表面可能出现裂纹、夹杂、斑块、麻点、氧化铁皮、划痕等多种缺陷。传统的人工目视检测存在效率低、漏检率高、标准不一致、数据难追溯等问题。随着机器视觉和深度学习技术的发展，基于卷积神经网络的自动缺陷检测成为工业质量控制的重要方向。

本项目面向工业质量控制场景，开发一套基于轻量化卷积神经网络的表面缺陷智能检测系统。系统采用 B/S 架构，支持工件图片上传、缺陷自动识别分类、检测结果可视化展示与历史数据追溯，旨在辅助人工质检，降低漏检率，提升质检效率与检测标准一致性。

1.3 术语与缩略语

术语/缩略语	全称	说明
CNN	Convolutional Neural Network	卷积神经网络，深度学习的核心模型之一
B/S	Browser/Server	浏览器/服务器架构，用户通过浏览器访问系统
API	Application Programming Interface	应用程序编程接口
REST	Representational State Transfer	表述性状态转移，一种 API 设计风格
SQLite	—	轻量级嵌入式关系型数据库
UI	User Interface	用户界面
SRS	Software Requirements Specification	软件需求规格说明书
Vibe Coding	—	AI 辅助人机协同编程方法
NEU-CLS	—	东北大学热轧钢表面缺陷数据集
ROI	Region of Interest	感兴趣区域

FPS	Frames Per Second	每秒处理帧数，衡量推理速度
-----	-------------------	---------------

表 1-1 术语与缩略语表

1.4 参考资料

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

[2] GB/T 8567-2006, 计算机软件文档编制规范.

[3] 张智海等. 制造智能技术基础[M]. 北京：清华大学出版社，2022.

[4] 杨杰. 人工智能基础[M]. 北京：机械工业出版社，2025.

[5] Song K, Yan Y. A noise robust method for surface defect detection of steel products[J]. IEEE TIM, 2013.

[6] He K, et al. Deep Residual Learning for Image Recognition[C]. CVPR, 2016.

[7] Flask 官方文档. <https://flask.palletsprojects.com/>

[8] PyTorch 官方文档. <https://pytorch.org/docs/>

[9] 本项目设计报告《基于轻量化卷积神经网络的工业零部件表面缺陷智能检测系统》.

2 任务概述

2.1 系统目标

本系统的总体目标是：开发一套基于轻量化卷积神经网络的工业零部件表面缺陷智能检测系统，实现从图片上传、缺陷自动识别分类到结果展示与数据追溯的完整业务闭环。具体目标包括：

- (1) 功能目标：支持6类典型表面缺陷（裂纹、夹杂、斑块、麻点、氧化铁皮、划痕）的自动识别与分类，检测结果包含类别和置信度；支持历史记录查询与统计分析。
- (2) 性能目标：单张图片推理时间不超过100ms（CPU环境），模型分类准确率不低于90%，系统支持并发检测请求。
- (3) 架构目标：采用完整的B/S四层架构（前端UI层、后端服务层、算法模块层、数据存储层），各层通过标准协议解耦，算法模块独立封装。
- (4) 技术目标：覆盖《制造智能技术》课程至少3个核心技术方向——工业机器视觉与图像处理、工业机器学习与智能分类识别、工业质量数据管理与追溯。
- (5) 部署目标：轻量化设计，模型参数量控制在20万以内，可在普通PC机CPU环境下运行，无需GPU支持，降低部署门槛。

【评分要点】系统目标部分老师会看目标是否可量化、可验证。本文档每个目标都有具体指标（如"推理时间不超过100ms"、"准确率不低于90%"），而不是模糊的"提高效率"。这是高需求文档的关键特征——每个需求都应是可测试的。

2.2 用户特征

用户类型	特征描述	使用频率	技术水平
质检操作员	质检人员，使用系统上传工件图片并查看检测结果	高频（每日多次）	低，只会用浏览器
质量管理员	查看历史检测记录、统计分析报表，进行质量追溯	中频（每日/每周）	中，能操作数据筛选
系统管理员	配置系统参数、管理模型权重、维护数据库	低频（按需）	高，懂技术配置
课程评审教师	演示评估、随机代码提问、检查系统完整性	一次性（答辩）	高，专业技术人员

表 2-1 用户特征分析

系统设计以质检操作员为主要用户，界面应简洁直观，操作步骤不超过3步完成一次检测。所有用户均通过浏览器访问系统，无需安装客户端软件。

2.3 运行环境

类别	配置要求	说明
服务器 OS	Windows 10/11 或 Linux (Ubuntu 20.04+)	跨平台支持

Python 版本	Python 3.8+	推荐 3.10
Web 框架	Flask 2.0+	轻量级 Web 框架
深度学习框架	PyTorch 1.10+	CPU 版本即可，支持 GPU 加速
图像处理	OpenCV 4.5+	图像读取与预处理
数据库	SQLite 3	Python 内置，无需额外安装
前端浏览器	Chrome 90+ / Edge 90+ / Firefox 88+	支持 ES6 和 Fetch API
硬件最低配置	CPU 双核 2.0GHz / 内存 4GB / 硬盘 2GB	可运行
硬件推荐配置	CPU 四核 3.0GHz / 内存 8GB / SSD	流畅运行

表 2-2 运行环境要求

2.4 条件与约束

- （1）数据约束：系统使用 NEU-CLS 公开数据集（6 类缺陷，每类 300 张）进行模型训练，不涉及企业真实数据，无数据隐私问题。
- （2）技术约束：模型须在 CPU 环境下可运行，推理时间可接受；系统须为 B/S 架构，不采用 C/S 架构。
- （3）开发约束：采用 Vibe Coding 方法，全程使用 AI 辅助开发，须如实披露 AI 工具使用情况；使用 Git 进行版本控制。
- （4）时间约束：项目开发周期为 10 天（含选题、数据、开发、调试、报告）。
- （5）安全约束：上传文件大小限制为 10MB，仅支持 JPG/PNG/BMP 格式；须防范路径穿越、SQL 注入等常见 Web 安全风险。

3 功能需求

3.1 功能划分

系统功能划分为四大模块，各模块的功能编号、名称和优先级如下表所示。优先级分为高（必须实现）、中（应该实现）、低（可以实现）三级。

功能编号	功能名称	所属模块	优先级	说明
FR-01	图片上传与检测	缺陷检测	高	上传工件图片，返回缺陷类别和置信度
FR-02	检测结果展示	缺陷检测	高	展示原图、预测类别、置信度、各类别概率
FR-03	历史记录查询	历史管理	高	按时间/类别筛选查询历史检测记录
FR-04	记录详情查看	历史管理	中	点击记录查看检测详情和原图
FR-05	缺陷分布统计	统计分析	中	统计各类缺陷数量，展示饼图
FR-06	检测趋势分析	统计分析	低	按时间展示检测数量趋势折线图
FR-07	模型权重管理	系统管理	中	加载/切换模型权重文件
FR-08	数据库初始化	系统管理	高	创建数据库和检测记录表
FR-09	文件格式校验	系统管理	高	校验上传文件格式和大小

表 3-1 系统功能划分

3.2 缺陷检测功能（FR-01、FR-02）

功能描述：用户在检测页面上传工件表面图片，系统自动进行图像预处理和 CNN 模型推理，返回缺陷类别、置信度和各类别概率，并在页面可视化展示。

输入：工件表面图片文件（JPG/PNG/BMP 格式，大小不超过 10MB）。

处理流程：

- (1) 系统校验上传文件格式和大小，不符合要求时返回错误提示；
- (2) 将图片保存到服务器 uploads 目录，文件名经 secure_filename 安全过滤；
- (3) 调用 DefectPredictor 推理模块：读取图片→灰度化→Resize 到 200×200→ToTensor→Normalize(mean=0.5, std=0.5)→CNN 前向推理→Softmax 输出概率；
- (4) 取概率最大的类别作为预测结果，将检测记录写入 SQLite 数据库；
- (5) 返回 JSON 格式的检测结果，前端渲染展示。

输出：JSON 对象，包含 predicted_class（类别编号 0-5）、predicted_name（类别中文名）、confidence（置信度 0-1）、confidence_percent（置信度百分比）、probabilities（6 类概率列表）。

异常处理：文件格式不支持→返回 400 错误；文件过大→返回 413 错误；图片读取失败→返回 500 错误并记录日志。

3.3 历史记录管理功能（FR-03、FR-04）

功能描述：系统自动保存每次检测记录，用户可按时间范围和缺陷类别筛选查询，支持分页浏览，点击记录可查看详情。

输入：筛选条件（起始日期、结束日期、缺陷类别）、页码、每页条数。

处理流程：（1）接收查询参数；（2）动态构建 SQL 查询语句（参数化查询防注入）；（3）执行查询并返回指定页的记录列表；（4）前端渲染为表格。

输出：JSON 对象，包含 records（记录列表）、page（当前页码）、total（总记录数）。

3.4 统计分析功能（FR-05、FR-06）

功能描述：对历史检测数据进行统计分析，以图表形式展示各类缺陷分布和检测趋势。

缺陷分布统计：按 predicted_name 分组统计各类缺陷的检测数量，用 Chart.js 绘制饼图，直观展示各类缺陷占比。

检测趋势分析：按日期（detect_time）分组统计每日检测数量，用 Chart.js 绘制折线图，展示检测量随时间的变化趋势。

3.5 系统管理功能（FR-07、FR-08、FR-09）

数据库初始化（FR-08）：运行 init_db.py 脚本，创建 defect_detection.db 数据库和 detection_records 表。表结构包含 id、image_path、predicted_class、predicted_name、confidence、detect_time 六个字段。使用 CREATE TABLE IF NOT EXISTS 避免重复创建报错。

模型权重管理（FR-07）：系统启动时从指定路径加载 best_model.pth 模型权重文件。管理员可通过修改配置或替换权重文件切换模型。模型加载失败时系统应给出明确错误提示。

文件格式校验（FR-09）：后端校验上传文件的扩展名（仅允许 .jpg/.jpeg/.png/.bmp）和文件大小（最大 10MB）。Flask 的 MAX_CONTENT_LENGTH 配置自动拦截超大文件。

3.6 用例描述

以下是系统核心用例的详细描述，采用 UML 用例规范格式。

用例 UC-01：上传图片进行缺陷检测

项目	内容
----	----

用例编号	UC-01
用例名称	上传图片进行缺陷检测
参与者	质检操作员
前置条件	系统已启动，模型已加载，数据库已初始化
基本流程	1. 用户点击“上传图片”按钮；2. 系统接收并验证图片格式；3. 系统调用缺陷检测模型进行推理；4. 系统返回检测结果；5. 用户查看检测结果。
备选流程	3a. 图片格式不支持（非JPG/PNG/BMP）→系统提示“仅支持JPG/PNG/BMP格式”；4b. 文件超过10MB→系统提示“文件大小不能超过10MB”；5a. 模型推理失败→系统提示“检测失败，请稍后再试”。
后置条件	检测记录已保存到数据库，页面展示检测结果
优先级	高

表 3-2 用例 UC-01 描述

用例 UC-02：查询历史检测记录

项目	内容
用例编号	UC-02
用例名称	查询历史检测记录
参与者	质量管理人员
前置条件	系统中已有检测记录
基本流程	1. 用户进入历史记录页面；2. 用户选择筛选条件（时间范围/缺陷类别）；3. 点击查询按钮；4. 系统返回符合条件的记录列表；5. 页面展示记录详情。
备选流程	2a. 不选筛选条件→返回全部记录；4a. 无符合条件记录→提示“暂无检测记录”
后置条件	无（只读操作）
优先级	高

表 3-3 用例 UC-02 描述

【评分要点】用例描述是需求文档的核心评分点。老师会看：用例是否完整（前置条件、基本流程、备选流程、后置条件齐全）、流程是否可执行、异常情况是否覆盖。本文档的用例严格按照UML规范编写，每个备选流程都有编号（如4a、4b），对应基本流程的第几步，这是专业写法。

4 非功能需求

4.1 性能需求

需求编号	需求描述	指标	验证方法
NFR-01	单张图片推理时间	≤ 100ms（CPU）	evaluate.py 计时统计
NFR-02	模型分类准确率	≥ 90%（测试集）	evaluate.py 评估
NFR-03	API 响应时间	≤ 500ms（含图片上传）	pytest 接口测试计时
NFR-04	并发处理能力	支持 ≥ 5 并发请求	压测工具测试
NFR-05	系统启动时间	≤ 10 秒（含模型加载）	手动计时
NFR-06	支持图片分辨率	≥ 50×50, ≤ 4096×4096	边界测试
NFR-07	模型参数量	≤ 200,000	count_parameters() 统计

表 4-1 性能需求

【编写说明】每个性能需求都有明确的量化指标和验证方法，这是"可验证需求"的体现。老师会看需求是否可测试——如果只写"系统要快"就无法验证，写"≤100ms"就可以用 evaluate.py 验证。

4.2 安全需求

- （1）文件上传安全（NFR-08）：仅允许 JPG/PNG/BMP 格式，文件大小限制 10MB；使用 secure_filename() 过滤文件名，防止路径穿越攻击；上传文件存储在独立目录，不执行。
- （2）数据库安全（NFR-09）：所有 SQL 查询使用参数化查询（? 占位符），禁止字符串拼接 SQL，防止 SQL 注入攻击；数据库文件权限设置为仅应用进程可读写。
- （3）输入校验（NFR-10）：所有用户输入（页码、筛选条件、文件名）在后端进行类型和范围校验，拒绝非法输入。
- （4）错误处理（NFR-11）：系统异常时返回友好错误提示，不暴露堆栈信息和内部路径；关键操作记录日志。
- （5）中文路径安全（NFR-12）：使用 np.fromfile + cv2.imdecode 读取图片，避免 OpenCV 在中文路径下读取失败导致的安全隐患。

4.3 可用性需求

- （1）易用性（NFR-13）：检测操作不超过 3 步（选择图片→点击检测→查看结果）；界面简洁直观，关键按钮醒目；检测结果用图表可视化，无需专业知识即可理解。
- （2）响应性（NFR-14）：检测过程中页面显示加载状态，不冻结；操作反馈时间不超过 1 秒（加载动画）。

(3) 容错性 (NFR-15) : 用户误操作 (如未选图片就点检测) 时给出明确提示, 不导致系统崩溃; 网络异常时提示重试。

(4) 可访问性 (NFR-16) : 支持主流浏览器 (Chrome/Edge/Firefox) ; 页面响应式布局, 适配不同屏幕尺寸。

4.4 可维护性需求

(1) 模块化设计 (NFR-17) : 算法模块 (model.py、inference.py)、数据模块 (dataset.py)、后端服务 (app.py)、数据库 (init_db.py) 独立封装, 模块间通过明确接口通信, 修改一个模块不影响其他模块。

(2) 代码规范 (NFR-18) : 代码有完整中文注释, 函数和类有 docstring; 命名规范 (类名大驼峰、函数名小写下划线、常量全大写) ; 遵循 PEP 8 规范。

(3) 可测试性 (NFR-19) : 提供 pytest 自动化测试脚本, 覆盖模型推理、API 接口、端到端流程; 关键函数可独立单元测试。

(4) 可配置性 (NFR-20) : 模型路径、上传目录、文件大小限制等参数可通过配置文件或代码顶部常量修改, 无需改动核心逻辑。

4.5 可移植性需求

(1) 跨平台 (NFR-21) : 系统可在 Windows 和 Linux 上运行, 不依赖特定操作系统 API; 路径处理使用 os.path.join() 而非硬编码分隔符。

(2) 模型可迁移 (NFR-22) : 模型权重保存为标准 .pth 格式, 可在不同机器上加载; 使用 map_location 参数支持 CPU/GPU 环境切换。

(3) 数据库可迁移 (NFR-23) : SQLite 数据库为单个 .db 文件, 可直接复制迁移; 表结构设计遵循第三范式, 便于后续迁移到 MySQL/PostgreSQL。

5 接口需求

5.1 用户接口

系统用户接口为 Web 浏览器界面，包含以下页面：

页面	URL	主要元素	用户操作
检测首页		图片上传区、检测按钮、结果展示区、概率饼图	上传图片、点击检测、查看结果
历史记录页	/history	筛选栏、记录表格、分页控件	筛选查询、翻页、查看详情
统计分析页	/stats	缺陷分布饼图、检测趋势折线图	查看统计图表、切换时间范围

表 5-1 用户接口页面

界面设计原则：（1）顶部导航栏统一，可在页面间快速切换；（2）检测结果用颜色区分（高置信度绿色、低置信度橙色）；（3）所有操作有明确的按钮和提示，无需阅读说明书即可使用。

5.2 软件接口

系统提供 RESTful API 接口，前端通过 HTTP 请求调用，返回 JSON 格式数据。

接口 1：缺陷检测接口

项目	内容
URL	/api/detect
方法	POST
Content-Type	multipart/form-data
请求参数	image: 文件对象（必填）
成功响应	HTTP 200, JSON: {id, image_path, predicted_class, predicted_name, confidence, confidence_percent, probabilities, class_names, class_name}
失败响应	HTTP 400: {error: "未上传图片"}; HTTP 413: 文件过大
调用示例	fetch("/api/detect", {method:"POST", body: formData})

表 5-2 缺陷检测 API

接口 2：历史记录查询接口

项目	内容
URL	/api/history
方法	GET
请求参数	page: 页码（默认1）; page_size: 每页条数（默认10）; class: 缺陷类别筛选（可选）
成功响应	HTTP 200, JSON: {records: [{id, image_path, predicted_class, predicted_name, confidence, detect_time}], page, total}
失败响应	HTTP 400: 参数错误

表 5-3 历史记录查询 API

接口 3：统计数据接口

项目	内容
URL	/api/stats
方法	GET
请求参数	start_date: 起始日期（可选）; end_date: 结束日期（可选）
成功响应	HTTP 200, JSON: {class_distribution: [{name, count}], daily_trend: [{date, count}]}

表 5-4 统计数据 API

内部软件接口

- （1）算法模块接口：DefectPredictor.predict(image_path) → dict，后端通过此接口调用模型推理，接口稳定，模型内部实现可替换。
- （2）数据库接口：使用 Python 内置 sqlite3 模块，通过 SQL 语句操作数据库。
- （3）模型文件接口：模型权重为 .pth 格式，通过 torch.load() 加载。

5.3 硬件接口

- 系统为纯软件系统，不直接操作硬件。间接硬件依赖包括：
- （1）图像采集设备：用户通过工业相机或手机拍摄工件图片，保存为 JPG/PNG/BMP 文件后上传系统。系统不直接控制相机。
 - （2）计算设备：服务器端使用 CPU 进行模型推理，支持 GPU 加速（如有）。PyTorch 自动检测可用设备，通过 torch.cuda.is_available() 判断。
 - （3）存储设备：上传图片和数据库文件存储在服务器硬盘上。

6 数据需求

6.1 数据字典

系统核心数据项的定义如下：

数据项	类型	长度/范围	说明	来源
image_path	TEXT	≤255字符	上传图片的保存路径（相对路径）	系统生成
predicted_class	INTEGER	0-5	预测缺陷类别编号	模型输出
predicted_name	TEXT	≤20字符	预测缺陷类别中文名称	类别映射
confidence	REAL	0.0-1.0	预测置信度	模型输出
detect_time	TEXT	19个字符	检测时间，格式 YYYY-MM-DD HH:MM:SS	系统时间
id	INTEGER	自增	记录唯一标识	数据库自增
probabilities	ARRAY	6个浮点数	6类缺陷的概率分布	模型输出

表 6-1 数据字典

6.2 数据库设计

系统使用 SQLite 数据库，包含一个核心表 detection_records（检测记录表）。

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY AUTOINCREMENT	记录唯一标识，自增主键
image_path	TEXT	NOT NULL	上传图片的保存路径
predicted_class	INTEGER	NOT NULL, CHECK(0-5)	预测类别编号
predicted_name	TEXT	NOT NULL	预测类别中文名称
confidence	REAL	NOT NULL, CHECK(0-1)	预测置信度
detect_time	TEXT	NOT NULL	检测时间

表 6-2 detection_records 表结构

建表 SQL 语句：

SQL：创建检测记录表

```
CREATE TABLE IF NOT EXISTS detection_records (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    image_path TEXT NOT NULL,  
    predicted_class INTEGER NOT NULL,  
    predicted_name TEXT NOT NULL,  
    confidence REAL NOT NULL,  
    detect_time TEXT NOT NULL  
);
```

数据库设计遵循第三范式（3NF）：每个字段不可再分，非主键字段完全依赖主键，不存在传递依赖。后续如需扩展（如用户表、模型版本表），可新增表并通过外键关联。

6.3 数据采集与处理

训练数据：使用 NEU-CLS 公开数据集，6 类缺陷，每类 300 张，共 1800 张 200×200 灰度图。数据按 70%/15%/15% 分层抽样划分为训练集、验证集、测试集。

数据预处理流程：

- （1）扫描数据集目录，生成包含图片路径和标签的 CSV 索引文件；
- （2）分层抽样划分，固定随机种子 42 保证可复现；
- （3）训练集数据增强：随机水平翻转、垂直翻转、旋转 $\pm 10^\circ$ ；
- （4）统一预处理：Resize 到 $200 \times 200 \rightarrow \text{ToTensor} \rightarrow \text{Normalize}(\text{mean}=0.5, \text{std}=0.5)$ 。

运行时数据：用户上传的图片保存在 uploads 目录，检测记录写入数据库。图片文件和数据库记录通过 image_path 字段关联。

数据备份：SQLite 数据库文件可直接复制备份；建议定期备份 defect_detection.db 和 uploads 目录。

7 其他需求

7.1 配置管理需求

- (1) 使用 Git 进行版本控制，代码托管在 GitHub 仓库；
- (2) 提交信息遵循规范格式（如 feat: 新增功能、fix: 修复bug、docs: 文档更新）；
- (3) 小步提交，每次提交对应一个完整的功能点或修复；
- (4) 保留完整的提交历史，便于追溯和回滚。

7.2 文档需求

文档名称	用途	交付状态
需求规格说明书	描述系统需求，指导设计与测试	本文档
设计报告	描述系统设计与实现细节	已完成
README.md	项目说明、安装运行指南	已完成
代码注释	函数级和行级中文注释	已完成
测试报告	自动化测试结果	随 tests/ 目录提供
AI 使用披露	AI 工具使用情况、prompt 策略、出错纠正	设计报告第六章

表 7-1 项目文档清单

7.3 培训需求

- (1) 质检操作员培训：约 10 分钟，包括系统登录、图片上传、结果查看、历史记录查询等基本操作。系统界面简洁，培训后即可独立使用。
- (2) 管理员培训：约 30 分钟，包括系统部署、模型权重替换、数据库备份、常见故障排查。
- (3) 在线帮助：系统各页面提供操作提示，README.md 包含完整的安装运行说明。

需求追溯说明

本文档中每个功能需求都有唯一编号（FR-XX），每个非功能需求也有唯一编号（NFR-XX）。在设计报告和测试文档中，可通过这些编号追溯需求的实现和验证情况。例如，FR-01（图片上传与检测）对应设计报告第四章的 /api/detect 接口实现，对应测试文档中的 T03-T05 测试用例。这种可追溯性是软件工程规范的要求，也是评分的加分项。

【评分要点】需求规格说明书的高分要点总结：（1）结构完整：严格按照 IEEE 830 标准，7 大章节齐全；（2）需求可验证：每个需求有编号、有量化指标、有验证方法；（3）用例规范：前置条件、基本流程、备选流程、后置条件完整；（4）接口详细：API 的 URL、方法、参数、响应格式都有明确描述；（5）数据完整：数据字典、表结构、建表 SQL 齐全；（6）无歧义：避免"大概""可能""尽量"等模糊表述。本文档全部满足以上要求，可作为课程设计提交物。

— 文档完 —